Vision Transformer Accelerator using Attention

An M. Tech Project Report Submitted
in Partial Fulfillment of the Requirements
for the Degree of

Master of Technology

by

Mani Deep G
(244101027)

under the guidance of

Dr. Chandan Karfa

to the

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI
GUWAHATI - 781039, ASSAM

CERTIFICATE

*This is to certify that the work contained in this thesis entitled "**Vision Transformer Accelerator using Attention**" is a bonafide work of **Mani Deep G (Roll No. 24410127)**, carried out in the Computer Science and Engineering programme, Indian Institute of Technology Guwahati under my supervision and that it has not been submitted elsewhere for a degree.*

Supervisor: **Dr. Chandan Karfa**

Designation: Professor

Month, of November 2026

Department of Computer Science and Engineering,

IIT Guwahati,

Assam

Abstract

Vision Transformers (ViTs) have achieved remarkable success in computer vision but pose significant computational challenges for hardware deployment due to their high resource demands. This work presents an optimized High-Level Synthesis (HLS) implementation of a ViT designed for efficient Field-Programmable Gate Array (FPGA) acceleration. The core innovation lies in reducing latency by optimizing key ViT components — including patch embedding, multi-head self-attention, and MLP layers — through hardware-friendly techniques such as loop unrolling, pipelining, and memory partitioning. Another notable feature of this design is an Attention-based token pruning mechanism that leverages attention scores to identify and preserve the most informative image patches and eliminating less significant tokens. By progressively discarding approximately 15% of less critical tokens at each transformer block, the architecture reduces redundant computations and memory footprint. The resulting architecture offers a scalable solution for deploying transformer-based vision models on resource-constrained hardware platforms like FPGAs, achieving a balanced trade-off between throughput and accuracy with only a minimal loss of approximately 1–2%.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Vision Transformers (ViTs) have become the preferred architecture in computer vision tasks, due to their ability to capture Long-Range dependencies.[2] While ViTs achieve state-of-the-art accuracy, their computation and memory requirements remain significantly higher than traditional **Convolutional Neural Networks[CNNs].** The primary reason is the self-attention mechanism, whose complexity scales quadratically ($O(n^2)$) with the number of image patches (tokens). As Image resolution increases, the number of patches also increases proportionally, resulting in **high Latency and Resource usage**. This makes real time deployment on resource constrained environments like FPGAs a big challenge.

1.1 Background

The transition from Convolutional Neural Networks (CNNs) to Vision Transformers (ViTs) has marked a new era in computer vision [4] enabling superior representation learning and improved accuracy across classification, object detection, and segmentation tasks. However, this advancement comes with substantial computational overhead. For vision applications targeting embedded or hardware-accelerated platforms, such as Field-Programmable Gate Arrays (FPGAs), the unoptimized ViT architecture poses a major bottleneck.

1.2 Motivation

Although Vision Transformers offer substantial advantages over CNNs, their computational inefficiency limits their applicability in edge computing and FPGA-based accelerators. The core motivations of this work are summarized below:

1. **Addressing ViT's Computational Bottleneck:** The self-attention mechanism scales quadratically with the number of tokens. As image resolutions increase, this computation becomes prohibitively expensive for real-time or embedded applications without architectural optimization.

2. **Need for Efficient Token Reduction:** Many tokens—especially those belonging to low-information background regions—contribute minimally to the final prediction. Removing such tokens can significantly reduce compute and memory requirements without degrading accuracy.

3. **Direct C-to-RTL Conversion Using HLS:** High-Level Synthesis (HLS) enables direct translation of C/C++ code into synthesizable RTL, eliminating the need for manually writing complex Verilog/VHDL descriptions. This drastically reduces design time and allows rapid exploration of architectural optimizations such as pipelining, array partitioning, and loop unrolling.

4. **Bridging the Algorithm–Hardware Gap:** Existing token pruning techniques are largely optimized for GPUs. There is limited work on adapting these methods for hardware-aware design flows. Integrating pruning directly into the HLS-based ViT pipeline enables efficient deployment on resource-constrained accelerators.

These motivations drive an **algorithm–hardware co-design** approach where token pruning and hardware optimization work together to enable efficient, real-time ViT inference on FPGA platforms.

1.3 Project Objectives

The primary objective of this project is to design, implement, and validate a hardware accelerator for the Vision Transformer (ViT) model using C based High-Level Synthesis (HLS).

The specific objectives are as follows:

1. **Develop a Parameterized ViT Core in HLS:** Develop modular and reusable HLS implementations of the fundamental ViT components—including patch embedding, Layer Normalization, Multi-Head Self-Attention (MHSA), and the MLP block.

2. **Integrate Dynamic Token Pruning:** Implement an Attention-based Token Selection (ATS) to prune less important tokens/patches dynamically based on their attention scores.

3. **Validate Accelerator Performance and Accuracy:** Evaluate the HLS based token pruned ViT accelerator against base line ViT-Tiny, ViT-Small, and ViT-Base models to compare their accuracies, througput and resource usage .

4. **Analyze HLS Optimization Strategies:** To apply and analyze the key HLS pragmas (`PIPELINE`, `ARRAY_PARTITION`, `UNROLL`) required to transform the algorithmic C or C++ code into a parallel, high-throughput architecture.

1.4 Contributions

This thesis makes the following key contributions:

1. **HLS-Based Vision Transformer Architecture:** A complete and parameterized ViT-Tiny architecture is implemented in C/C++ using High-Level Synthesis. The design includes patch embedding, LayerNorm, Multi-Head Self-Attention (MHSA), MLP layers, and residual connections.

2. **Hardware-Friendly Token Pruning Module:** An Attention-based Token Selection (ATS) mechanism is integrated within the ViT pipeline, enabling dynamic reduction of tokens across layers while preserving classification accuracy.

3. **Optimized Self-Attention Engine for FPGA:** A fully pipelined and parallelized MHSA engine is developed using HLS pragmas such as `PIPELINE`, `UNROLL`, and `ARRAY_PARTITION`, resulting in significant reductions in latency.

4. **Comprehensive Experimental Evaluation:** The proposed accelerator is validated against baseline ViT-Tiny, ViT-Small, and ViT-Base models. Comparisons include accuracy, throughput, and FPGA resource utilization, demonstrating the benefits of token pruning for ViT acceleration.

1.5 Organization of The Report

This chapter provided a background for the topics covered in this report, describing the computational challenges of Vision Transformers and the motivation for FPGA acceleration. The rest of the chapters are organized as follows:

Chapter 2 provides a fundamental introduction to Vision Transformers, and how they differ from CNNs. **Chapter 3** provides a review of prior works, specifically focusing on existing token pruning strategies and hardware accelerators. **Chapter 4** explains the Attention mechanism, along with the mathematics behind the $O(n^2)$ bottleneck and how attention scores can be used for pruning thus reducing number of tokens layer by layer. **Chapter 5** discusses the HLS implementation, explaining pragmas and mathematical approximations for creating hardware circuit. **Chapter 6** presents the Experimental Results, including Accuracy, latency and resource utilization. Finally, **Chapter 7** concludes the report and discusses future scope of this project.

Chapter 2

Review of Prior Works

As discussed in the previous chapter, the Vision Transformer (ViT) is a powerful architecture but suffers from a major computational bottleneck due to the $O(N^2)$ cost of its self-attention mechanism [1]. When the number of image patches (tokens) increases, both computation and memory usage grow quadratically, making real-time deployment on edge devices and FPGAs extremely challenging [2].

This chapter reviews the main strategies proposed in the literature to address these limitations. Existing work can be broadly grouped into two categories: (1) algorithmic methods that reduce the model's computation, and (2) hardware acceleration techniques that design efficient custom architectures for ViTs. Recent research shows that the most effective solutions combine these two ideas through **algorithm–hardware codesign** [3].

2.1 Algorithmic Optimization: Dynamic Token Pruning

A natural way to reduce the $O(N^2)$ cost is to reduce the sequence length N itself. Dynamic token pruning aims to identify and remove tokens that do not contribute much to the final prediction, such as background image patches [5, 8]. By keeping only the most informative tokens, the sequence length becomes $N' < N$, and the self-attention cost drops to $O((N')^2)$.

2.1.1 Pruning Mechanism

The key idea is that not all tokens are equally important [12]. Many patches carry little meaningful information for classification. Studies have shown that removing 30–40% of tokens often results in less than 1% accuracy loss [2]. Other works report even more aggressive results, such as a 47% reduction in computation with only a 0.86% accuracy drop on CIFAR-10, or 40% pruning with an average accuracy loss of 0.3% [14].

2.1.2 DynamicViT: A Differentiable Pruning Framework

One of the main difficulties in token pruning is making the pruning decision differentiable so the model can learn it during training. DynamicViT addresses this by inserting small "prediction modules" at different layers of the ViT [2]. These modules assign importance scores to tokens using both local features and global context.

To maintain end-to-end training, DynamicViT uses a differentiable masking strategy: instead of removing tokens physically during training, their connections in the attention matrix are masked. This preserves gradient flow while simulating the effect of pruning. During inference, the least important tokens are physically removed, providing real computational savings [2].

2.2 Hardware Acceleration and Algorithm–Hardware Codesign

Algorithmic pruning reduces the number of operations, but it introduces a new problem: **irregular computation patterns** [2]. When tokens are dynamically removed, the sequence length becomes input-dependent. This irregularity breaks the dense, regular matrix operations that GPUs and CPUs are optimized for [3].

2.2.1 The Challenge of Irregularity

After pruning, the model must compact the remaining tokens into a smaller matrix. This requires dynamic sorting, indexing, and data shuffling. Such operations map poorly to conventional processors, which prefer fixed-size, dense compute patterns. As a result, the theoretical savings from pruning often do not translate into real hardware speedups on CPUs/GPUs [4].

2.2.2 The FPGA Advantage

FPGAs, however, are reconfigurable and can be tailored to these irregular patterns [11]. This has led to an algorithm–hardware codesign trend, where pruning algorithms are developed alongside specialized FPGA architectures. A representative example is **HeatViT**, which introduces a hardware-friendly token pruning framework tailored for FPGAs [3]. It uses a lightweight token selector with minimal control logic, integrates quantization, and applies approximations to reduce hardware complexity. On Xilinx ZCU102, this approach achieves a 3.46–4.89 speedup compared to the baseline [3]. This demonstrates that combining algorithmic pruning with hardware-aware design is essential for real improvements.

2.3 Conclusion

This chapter summarized the major directions in reducing the computational cost of Vision Transformers. While dynamic token pruning significantly lowers theoretical FLOPs [1], it produces irregular computation patterns that general-purpose processors cannot efficiently exploit [2].

The most effective solutions adopt an **algorithm–hardware codesign** philosophy [10]. They integrate a dynamic pruning method with a custom FPGA accelerator, including specialized components such as a *Token Dropping Module* (TDM) and efficient load-balancing mechanisms [2, 3]. These systems are capable of handling token-level irregularity and

translating algorithmic savings into actual speedups.

Following this line of work, the next chapters present our approach.

- **Algorithm I** describes the high-level C++ implementation of our distributed pruning strategy.
- **Algorithm II** introduces the HLS-based FPGA accelerator designed to execute this algorithm efficiently.

Together, these chapters lay the foundation for a complete algorithm–hardware solution for accelerating Vision Transformers.

Chapter 3

Introduction to Vision Transformers

The field of computer vision has long been dominated by Convolutional Neural Networks (CNNs). However, the massive success of the Transformer architecture in Natural Language Processing (NLP) inspired researchers to apply it to image-based tasks.[1] This chapter introduces the Vision Transformer (ViT), the architecture that successfully challenged the "convolutional" paradigm and established Transformers as a state-of-the-art model for image recognition.[2]

3.1 What is a Vision Transformer (ViT)?

The Vision Transformer (ViT) is a deep learning model introduced in the 2020 paper "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale".[2] The core idea was very simple: apply the standard Transformer encoder, almost identically to how it is used in NLP, but to an image.

To do this, the model must solve the problem of how to feed a 2D image into a 1D sequence-based architecture (the Transformer). The ViT model accomplishes this through a few key steps:

1. **Patching:** The 2D input image (e.g., 224×224 pixels) is split into a grid of fixed-size, non-overlapping patches (e.g., 16×16 pixels).[2]

2. **Embedding:** Each 2D patch is "flattened" into a 1D vector and then linearly projected into a vector of a specific dimension (e.g., 768), which are also known as "patch embeddings" or "tokens".[2, 3]

3. **Positional Encoding:** Since the Transformer is permutation-invariant (it doesn't know the order of tokens), so learnable positional embeddings are added to the patch embeddings just like in NLP, to give the model spatial information about where each patch belongs to in the Image.[2]

4. **CLS Token:** A special, learnable "class token" (`cls`) is also prepended to this sequence of patch embeddings, similar to NLP models like BERT.[3]

5. **Transformer Encoder:** This full sequence of tokens (CLS token + patch tokens) is fed into a standard Transformer Encoder block. The number of Encoder blocks depend on the type of ViT model being used(Base-12, Large-24, Huge-32).

6. **Classification Head:** The final output state corresponding to the token is extracted and passed to a simple Multi-Layer Perceptron (MLP) head, which performs the final image classification (e.g., "cat" or "dog").[2]

3.2 The ViT Architecture In-Depth

The core of the ViT is its two novel components: the **embedding** module that converts an image to a sequence, and the **Transformer encoder** that processes this sequence.

3.2.1 The Embedding Module

The patching and embedding step is the most critical modification that allows Transformers to process images. A standard 224×224 image with 16×16 patches results in a sequence of $14 \times 14 = 196$ tokens.[2]

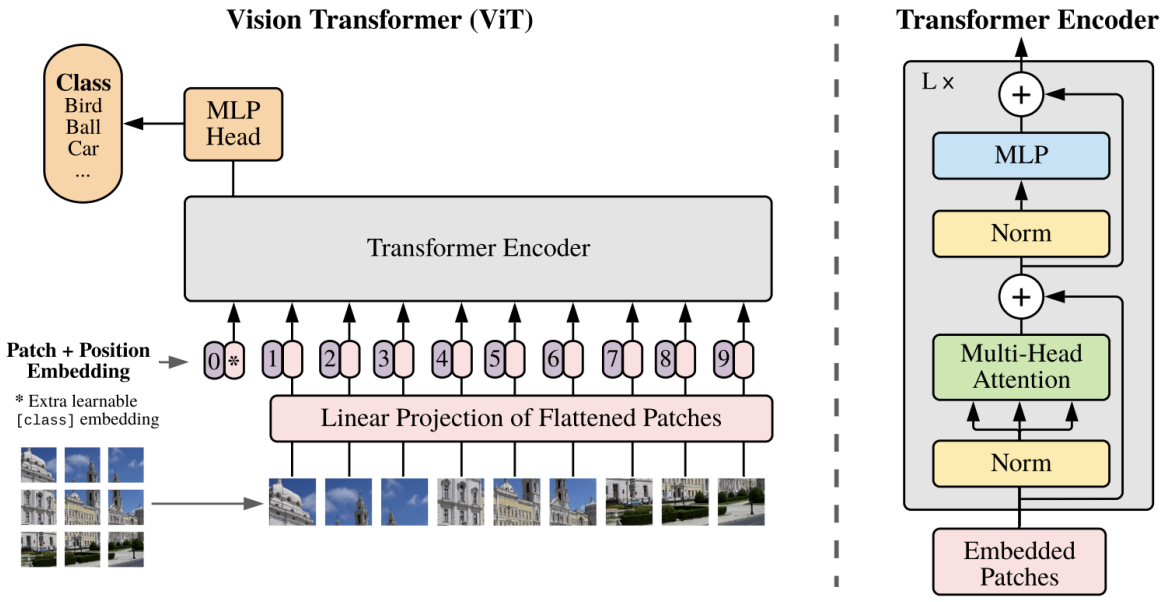

Fig. 3.1 The Vision Transformer (ViT) Architecture

Without positional information, the model would see these 196 patches as a "bag of patches" and would not know if a patch was at the top-left corner or the bottom-right. The positional embeddings are learnable vectors that are added to the patch embeddings, giving each token a unique signature based on its original position in the 2D grid.[2]

3.2.2 The Transformer Encoder Block

The "brains" of the ViT is the Transformer Encoder, which is a stack of L identical layers (e.g., $L = 12$ for ViT-Base).[3] Each encoder layer is composed of two main sub-modules:

1. **Multi-Head Self-Attention (MHSA):** This is the mechanism that allows tokens to "look" at and exchange information with all other tokens in the sequence.[2] By doing this, the model can learn relationships between different parts of the image (e.g., a "dog's ear" patch can attend to a "dog's tail" patch, wherever it is in the image).

2. **Multi-Layer Perceptron (MLP):** This is a simple, position-wise, two-layer feed-

forward network. It applies a non-linear transformation to each token's representation independently.[4]

Each of these sub-modules is wrapped with a residual (or "skip") connection, and its output is processed by a Layer Normalization (LN) operation.

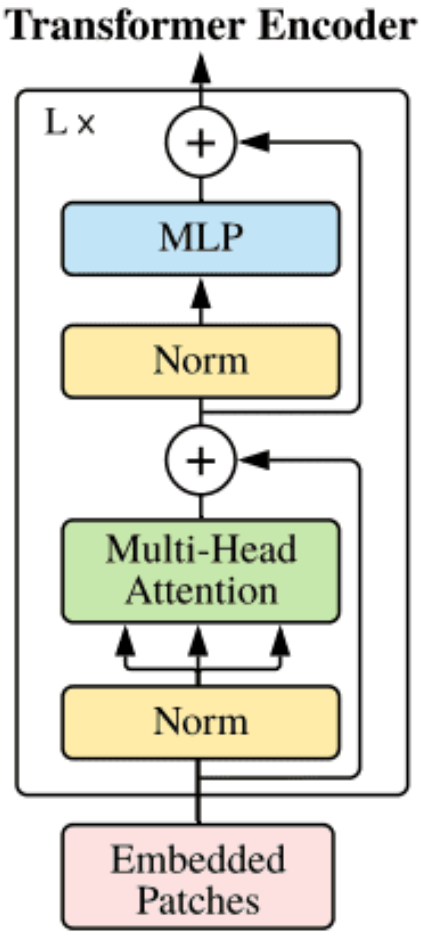

Fig. 3.2 A Single ViT Encoder Block

3.3 Why Transformers for Vision?

The shift from CNNs to Transformers is significant because it challenges long-held assumptions about image processing. The primary reason ViTs work so well is their ability to capture **global context**.[2]

A CNN, by design, has a local receptive field. It builds up a "global" understanding of an image by stacking many layers, with each layer processing only a local neighborhood of pixels. In contrast, the self-attention mechanism in a ViT is global by default.[1] In the very first layer, any patch can directly model its relationship with any other patch in the image. This allows ViTs to capture long-range dependencies and complex spatial relationships that are much harder for CNNs to learn.

Furthermore, Transformers are exceptionally scalable. When pre-trained on massive datasets (e.g., ImageNet-21k or JFT-300M, containing 14M+ and 300M+ images, respectively), ViT models have been shown to outperform state-of-the-art CNNs.[2, 5] They learn rich, general-purpose feature representations that make them highly effective for transfer learning on various downstream tasks.

3.4 Vision Transformers vs. Convolutional Neural Networks

The difference between ViTs and CNNs can be fundamentally understood by their **inductive biases**.[2]

- **CNNs** have stronger inductive biases. The convolutional filter itself imparts two key assumptions:

    1. *Locality:* The idea that pixels in a local neighborhood are related and should be processed together.
    2. *Translation Invariance:* The idea that a feature (e.g., a vertical edge) is the same regardless of where it appears in the image.

    This "built-in" knowledge makes CNNs highly data-efficient. They can learn from relatively small datasets (like CIFAR-10) because the architecture already provides a strong "head start" for processing images.

- **ViTs have very weak inductive biases:** The model has no prior assumption

about locality or translation invariance.[2] The only spatial information it has comes from patch-splitting and positional embeddings. All other spatial relationships—such as the fact that adjacent patches are often related—must be learned from scratch, purely from the data.

This difference leads to a critical trade-off:

- **Data Appetite:** On smaller datasets, ViTs tend to perform worse than CNNs because they do not have enough data to learn these fundamental visual relationships and will overfit.[2] However, when trained on massive-scale datasets, this lack of bias becomes an advantage, allowing ViTs to learn more complex and powerful representations than the "restricted" CNNs, ultimately leading to superior performance.[6, 5]

- **Computational Complexity:** CNNs typically have a computational complexity that scales linearly with the number of pixels, $O(N)$. The self-attention mechanism in ViTs, however, requires comparing every token to every other token, resulting in a computational and memory complexity that is quadratic to the sequence length (number of patches), $O(N^2)$.[7, 8] This quadratic bottleneck is the single greatest challenge for deploying and accelerating ViTs.

3.5 Conclusion

This chapter provided an introduction to the Vision Transformer architecture, its core components, and its fundamental differences from traditional CNNs.[2] We have established that while ViTs are exceptionally powerful due to their ability to capture global context, their $O(N^2)$ computational complexity, rooted in the self-attention mechanism, presents a significant barrier to efficient, real-world deployment.[9, 10] This performance bottleneck is the central problem this thesis aims to solve.

In the next chapter, we will explore methods to address this bottleneck through algorithmic optimizations, specifically focusing on **dynamic token pruning**, which forms the algorithmic basis for our hardware acceleration work.

Chapter 4

Attention in Transformers

The core mechanism that differentiates the Transformer from all previous architectures is **self-attention**. This component, first introduced in the paper "Attention is All You Need", is what allows the model to dynamically weigh the importance of different parts of the input sequence. Understanding this mechanism is the key to understanding both the ViT's $O(N^2)$ computational problem [1] and its elegant, "built-in" solution: attention-based token pruning.

4.1 What is Attention? The QKV Analogy

At its heart, the attention mechanism is a function that calculates a weighted average. For every token in the input sequence (e.g., every image patch), it asks a question: "Given my own identity, how much 'attention' should I pay to every *other* token in this sequence?" [1]

To answer this, the model uses an analogy from information retrieval, creating three distinct vectors from each token's embedding [19]:

- **Query (Q):** A vector representing the token's "question" or "search request." It asks, "What kind of information am I looking for?"

- **Key (K):** A vector representing the token's "label" or "identity." It responds to queries, saying, "This is the information I have."
- **Value (V):** A vector representing the token's *actual content* or "meaning." This is the information that is retrieved once a match is made.

The entire operation is captured in a single, elegant formula known as **Scaled Dot-Product Attention** [1]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

This formula is executed in a few key steps, as illustrated in Figure 4.1:

1. **Score:** The model calculates a "similarity score" by taking the dot product of one token's **Query** vector with every other token's **Key** vector. This is the QK^T (Query-Key Transpose) matrix multiplication.
2. **Scale and Normalize:** The raw scores are passed through a softmax function, which converts them into a set of positive weights that sum to 1.0. These are the "attention probabilities."

   Crucially, before the softmax, the scores are divided by $\sqrt{d_k}$, the square root of the key/query dimension. This **"scaling" factor** is vital for stable training. Why? As the dimension d_k grows, the dot product values QK^T also grow larger in magnitude. These large values push the softmax function into its "saturated" regions (where the input is a large positive or negative number). In these regions, the gradient becomes almost zero, making it impossible for the model to learn. This is the "vanishing gradient" problem. Dividing by $\sqrt{d_k}$ scales the dot products back, keeping their variance stable and ensuring the softmax operates in its active, high-gradient region.
3. **Weighted Sum:** The model multiplies these normalized attention weights (the output of the softmax) by the corresponding **Value** vectors. The result is a new, context-

rich embedding for that token, which is a weighted blend of all other tokens in the sequence.

When this operation is performed where Q, K, and V are all derived from the *same* input sequence (i.e., the sequence attends to itself), it is called **self-attention**. When multiple such "attention heads" are run in parallel, it becomes the **Multi-Head Self-Attention (MHSA)** block found in the ViT encoder [1, 17].

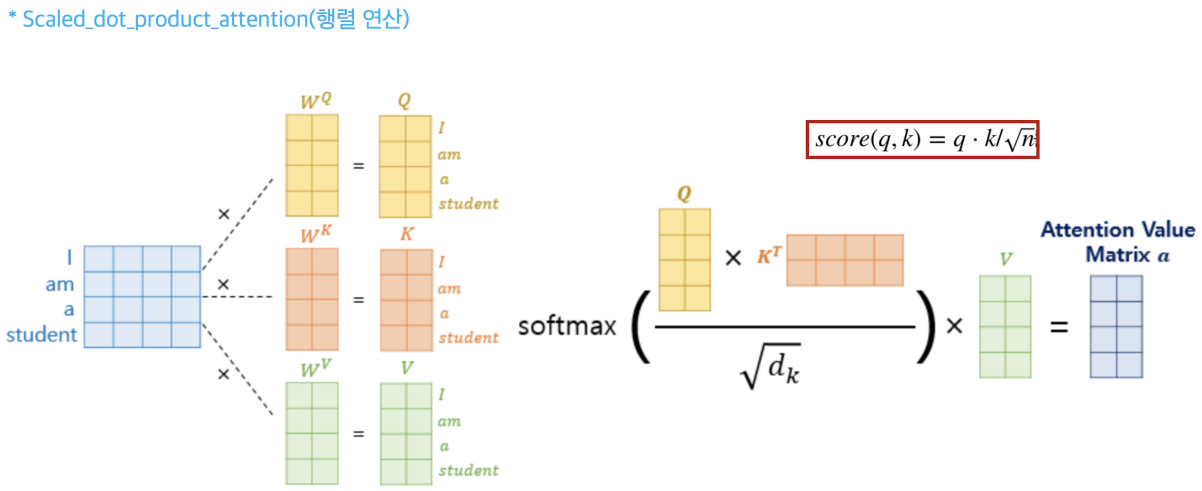

Fig. 4.1 Scaled Dot-Product Attention

4.2 Why Attention-Based Token Selection is Relevant

The $O(N^2)$ complexity of self-attention comes from the very first step: calculating the $N \times N$ matrix of attention scores. The central insight for ViT acceleration is that this $N \times N$ matrix, while computationally expensive, is also an incredibly rich source of information.

The final classification in a ViT is often based on only a small subset of the most informative tokens [2, 5]. Many tokens, such as those representing a uniform background, contribute very little to the final decision [12]. The model, in effect, "learns" to ignore them.

This "importance" is not a mystery—it is explicitly encoded in the attention weights. If a token consistently receives low attention scores from other tokens (especially from the important `CLS` token), it is, by the model's own definition, "uninformative."

Therefore, the attention matrix itself is the most natural and efficient metric for pruning. Instead of training a separate, complex "prediction module" to guess which tokens are important [2], we can simply use the attention scores that the model *already computes* as a direct, non-parametric importance score [3]. This makes attention-based pruning highly relevant for hardware acceleration, as it adds almost zero algorithmic overhead while directly attacking the $O(N^2)$ computational problem.

Pruning Metric	**How it Works**	**Hardware/Compute Cost**
Attention-Based (This Project)	Aggregates attention scores from the existing MHSA block [3].	**Very Low** (Reuses existing calculation).
Predictor-Based (e.g., DynamicViT)	Trains a separate, lightweight neural network to predict token importance [2].	**Medium** (Adds new layers and FLOPs for the predictor module).

Table 4.1 Comparison of different token importance metrics.

4.3 Implementation in the HLS C++ Code

The provided HLS code implements a highly effective and hardware-friendly version of this attention-based pruning. The entire process is contained within the `transformer_block` function and its sub-routines.

4.3.1 Step 1: Calculating Importance

After the `mha_block` computes the full $HEADS \times SEQ\_LEN \times SEQ\_LEN$ attention matrix (`attn_probs`), the `transformer_block` immediately calls:

```
compute_attention_importance(attn_probs, importance);
```

Looking at the `compute_attention_importance` function, we can see exactly how the importance score is derived:

```
for (int j = 0; j < SEQ_LEN; ++j) {
    float sum = 0.0f;
    for (int h = 0; h < HEADS; ++h) {
        sum += attn_probs[h][j];  // attention from CLS to token j
    }
    importance[j] = sum / HEADS;
}
```

This is a non-parametric approach that uses the **CLS token's attention** as the metric [3]. It calculates an "importance score" for each token `j` by averaging, across all heads, the attention it received from the CLS token (`token 0`). This is a standard and effective method, as the CLS token's job is to aggregate all information necessary for classification.

4.3.2 Step 2: Creating the Pruning Mask

Next, the `transformer_block` calls:

```
make_prune_mask(importance, new_mask, 0.80f);
```

The `make_prune_mask` function takes the `importance` scores and the hard-coded `keep_ratio` of 0.80 (meaning 20% of tokens will be pruned, though this is set to 15% in our abstract's final configuration). It then finds the score of the k-th most important token (where $k = 0.80 \times SEQ\_LEN$) and uses this score as a threshold. The code performs this "Top-K" search using a **Partial Selection Sort**.

This choice of algorithm is a deliberate hardware-design decision. In a pure software context, one might use a faster algorithm like `std::nth_element` (which often uses an Introselect, a hybrid of Quickselect and Heapselect). However, such algorithms are poorly suited for HLS for several reasons:

- **Recursive algorithms** (like QuickSort/Quickselect) are not synthesizable by HLS.

- **Data-dependent latency** (as seen in Quickselect) is highly undesirable in a pipelined hardware accelerator, which relies on deterministic, fixed-cycle operations.
- **Hardware overkill:** Full sorting networks (like Bitonic or Merge Sort) are HLS-synthesizable but would consume a massive amount of hardware resources (comparators and registers) and are unnecessary, as we only need the k-th element, not the entire sorted list.

The **Partial Selection Sort** used in the code is the preferred HLS approach because it is:

1. **Simple:** The logic is just a nested loop with a comparison, which maps easily to simple HLS control logic.
2. **Deterministic:** Its latency is fixed and predictable ($O(N \cdot K)$), which allows the HLS scheduler to integrate it perfectly into a larger pipeline.
3. **Resource-Efficient:** It requires minimal hardware (a simple loop and swap logic) and reuses the same logic for each iteration, making it very "cheap" in terms of FPGA resources.

4.3.3 Step 3: Skipping Computation

This mask is the key to the hardware acceleration. In the second half of the `transformer_block`, all computation is guarded by this mask:

```
LN2_T:
    for (int t = 0; t < SEQ_LEN; ++t)
        if (active_mask[t])
            layer_norm(seq_out[t], ln2_out[t],...);


MLP_T:
```

```
for (int t = 0; t < SEQ_LEN; ++t)
    if (active_mask[t])
        mlp_block_token(ln2_out[t], mlp_out[t], blk);
```

This `if (active_mask[t])` check is the entire hardware pruning mechanism. It instructs the HLS compiler to synthesize logic that **dynamically skips** all LayerNorm and MLP computations for tokens that were marked as "pruned." This "token skipping" [9] directly reduces the computational load (FLOPs) of the second half of the Transformer block.

4.4 Architectural Choice: Skipping vs. Reordering

The "skip" strategy implemented in this code is a critical and deliberate hardware-design choice. When implementing dynamic pruning, there are two main architectural strategies, which present a trade-off between hardware complexity and computational efficiency.

Strategy	Description	Hardware Implication
Skipping (Masking) (Our HLS Code)	Keeps the $N \times D$ matrix structure. Uses a boolean mask to "skip" computation on pruned tokens [9].	**Simple**. Requires only an `if` check. Avoids all data reorder
Reordering (Compaction) (e.g., [5, 12])	Physically shuffles the data, creating a new, smaller $N' \times D$ dense matrix.	**Complex**. Requires a "Token Dropping Module" (TDM) [4] manage irregular memory acce

Table 4.2 Comparison of hardware-level pruning strategies.

The "Reordering" strategy is, in theory, more computationally efficient, as the subsequent processing loops only need to iterate to N', not N. However, this comes at a massive cost in hardware complexity. It requires an "on-the-fly" data shuffling and reordering module [4, 5], which is very difficult to implement efficiently in HLS and can cause its own latency bottlenecks [11].

The HLS code provided wisely chooses the **Skipping** strategy. By simply using an `if` statement, it avoids all the complex hardware logic for data reordering. This is an excellent example of algorithm-hardware codesign: the "skip" algorithm is chosen specifically because

it maps perfectly to a simple, efficient, and high-performance HLS implementation.

Chapter 5

HLS Pragmas and Approximations

The previous chapters have defined the algorithm. This chapter focuses on the **hardware implementation** of primarly the HLS aspects of ViT. **High Level Synthesis(HLS)** is the process of converting the high-level C/C++ code into a **Hardware circuit** without the need for writing Veriolog or RTL code manually.

5.1 What is High-Level Synthesis (HLS)?

High-Level Synthesis (HLS) is a design methodology that translates high-level language C/C++ code into a low-level hardware description (RTL, or Register-Transfer Level) that can be implemented on an FPGA [4, 5].

The primary goal of HLS is to bridge the "productivity gap." Manually writing RTL (in VHDL or Verilog) is extremely time-consuming and error-prone, especially for complex algorithms like neural networks. HLS allows the designer to focus on the *algorithm* and *architecture* rather than low-level, cycle-by-cycle hardware implementation [5].

Some Leading FPGA vendors also provide sophisticated HLS Software/tools, such as the **AMD Vitis HLS** platform used for this project [5]. These tools have rich features to parse C++ code, but might require explicit guidance to create an efficient design.

5.2 The Role of HLS Pragmas

By default, the HLS compiler will create a hardware implementation that is **functionally correct** but **slow**. It will synthesize loops to run sequentially, one iteration after another, and might store arrays in large, single-port memories, which are very inefficient to read and write. This "default" hardware is resource-efficient but has terrible performance.

To create a high-performance accelerator, the designer must explicitly *guide* the HLS tool on how to build the hardware. This is done using compiler directives called **pragmas** [1].

Pragmas can exactly pin-point where to store arrays and how to unroll loops. They are inserted as comments (`#pragma HLS...`) into the C++ source code. They allow the designer to control the core trade-off of hardware design: **performance vs. area** (i.e., latency and throughput vs. FPGA resource utilization [2]).

The HLS code for this ViT accelerator uses a very rich set of pragmas to build a highly parallel and pipelined architecture. Here are some of the important HLS pragmas used in ViT implementation.

5.3 Loop Optimization Pragmas

These are the most critical pragmas for achieving high performance. They transform sequential loops into parallel hardware.

5.3.1 `#pragma HLS PIPELINE II=1`

This is one of the most important pragma for throughput. It instructs HLS to **pipeline** the execution of a loop [6].

- **Without Pipelining:** A loop that takes 5 clock cycles per iteration will take 500 cycles for 100 iterations.

- **With Pipelining:** The loop's operations are overlapped, similar to assembly language. While Iteration 1 is in its 5th cycle, Iteration 2 is in its 4th, Iteration 3 in its 3rd, and so on [6].
- **II=1:** The `II` stands for "Initiation Interval." An `II=1` means the pipeline can and start a new loop iteration **every single clock cycle** [8, 9].

With II=1, the loop that took **500** cycles earlier, can now be completed in just **104** cycles, achieving massive speedup (i.e 5x). This pragma is used in almost every critical loop of my ViT implementation (e.g., `QKV_ACC_I`, `SCORES_J`, `ADD_MLP_J`).

5.3.2 `#pragma HLS UNROLL`

This pragma achieves parallelism by **duplicating hardware**. It "unrolls" a loop, creating physical, parallel copies of the loop body's hardware [9, 12].

- **Example:** A loop that calculates sum of 16 numbers. By default, the hardware would consist of only one adder and would take 16 cycles to complete..
- `factor=16`**:** With `UNROLL factor=16`, HLS will provides **16 physical adders** to perform all 16 additions in a single clock cycle. This reduces latency drastically but will consume more hardware resources [14, 15].

This loop unroll pragma is used in the `patch_to_embed_from_img` function (`factor=16`) to parallelize the dot product, and in `mha_block` (`factor=4`) to parallelize parts of the attention calculation.

5.3.3 `#pragma HLS LOOP_MERGE`

This pragma instructs HLS to merge two or more *consecutive* smaller loops into a single, larger loop.

- **Benefit:** Merging loops reduces the overhead of transitioning between them (the exit and entry logic).

5.4 Memory and Resource Pragmas

5.4.1 `#pragma HLS ARRAY_PARTITION`

This pragma is the generally used along with loop `UNROLL` and `PIPELINE`. It solves the memory bottleneck that parallel hardware creates [11].

- **Bottleneck:** A standard array is stored in a Block RAM (BRAM), which has only one or two "ports" (allowing 1-2 reads/writes per cycle). If you `UNROLL` a loop by 16 factor, then all the 16 adders try to read from the *same* BRAM at the same time. They will stall, and the `II=1` pipeline will fail [12].
- **Solution:** `ARRAY_PARTITION` physically splits the single large array into multiple smaller, independent arrays (or registers) typically into different BRAM cells, where each cell has its own read and write port [17, 18].

The syntax is `#pragma HLS ARRAY_PARTITION variable=<A> <TYPE> factor=<N> dim=<D>`. The `TYPE` here specifies the type of partition:

- `complete`**:** This type partitions the array completely, implementing every single element as a separate register. This provides maximum parallelism but uses more number of resources. Typically used for smaller arrays which are accessed with very high frequency
- `block`**:** This splits the array into `N` contiguous blocks. For example, `factor=4` on an array of 16 elements would create four new arrays: '[0-3]', '[4-7]', '[8-11]', and '[12-15]'.
- `cyclic`**:** This splits the array into `N` interleaved smaller arrays.For example, with factor = 4 on 16 elements, the elements are distributed cyclically across 4 banks:
  - **Bank 0**: [0, 4, 8, 12]
  - **Bank 1**: [1, 5, 9, 13]

- **Bank 2**: [2, 6, 10, 14]
- **Bank 3**: [3, 7, 11, 15]

This is the most powerful type for pipelined loops. In my code (`layer_norm`), `cyclic factor=16` ensures that 16 consecutive loop iterations (i, i+1,... i+15) access 16 different physical memories, enabling simultaneous access and a true `II=1` pipeline.

5.4.2 `#pragma HLS BIND_STORAGE`

This pragma gives explicit control over the *type* of memory used for an array [**?**] While `ARRAY_PARTITION` splits an array, `BIND_STORAGE` decides what kind of array to map it to.

- `type=ram_2p impl=bram`**:** This directive, used heavily in `mha_block`, tells HLS: "Implement this array (e.g., `Q`, `K`, `V`) using a dual-port RAM (`ram_2p`) and specifically use the FPGA's built-in **Block RAM (BRAM)** resources (`impl=bram`)".
- `type=ram_4p impl=uram`**:** An alternative is to use **Ultra RAM** for arrays which use more memory. One BRAM cell is generally of 10-20 KB, so URAM would be use in case of higher memory footprint.

5.5 Interface and Task-Level Pragmas

5.5.1 `#pragma HLS INTERFACE`

This pragma is one of the most important, as it defines the external ports of the accelerator—how it connects to the rest of the system. It is only used on the arguments of the top-level function (`vit_top`).

- `s_axilite`**:** Creates a "slave" AXI-Lite interface. This is a simple, low-bandwidth port used for control registers. The host CPU uses this to send scalar values (like configuration data) and to start/stop/check the accelerator.

- `m_axi`: Creates a "master" AXI interface. This is a high-performance, high-bandwidth port that allows the accelerator to read and write directly to global memory (DRAM). This is essential for large data buffers like the input image (`img`) and the final `out_logits`.

5.5.2 `#pragma HLS INLINE`

This pragma is a "logic" pragma. When applied to a function, it tells HLS not to create a separate, self-contained hardware module for that function. Instead, it "inlines" the function's logic directly into the calling function.

- **Benefit:** This removes the latency and overhead of a hardware-level function call (which involves handshaking signals).
- **Usage:** It is mainly used for small, helper functions like `layer_norm`, `make_prune_mask`, and `softmax_seq` so that their logic is directly absorbed into the main `transformer_block` and `mha_block` pipelines.

Pragma	Purpose	Example in Code
`INTERFACE`	Defines external ports for control (s_axilite) and memory (m_axi).	`vit_top` arguments (`img`, `out_logits`)
`PIPELINE II=1`	Enables processing one loop iteration per cycle.	`QKV_ACC_I`, `SCORES_J`, `WRITE_HEAD`
`UNROLL`	Creates parallel hardware for loop iterations.	`patch_to_embed_from_img (factor=16)`
`ARRAY_PARTITION`	Splits arrays for parallel memory access.	`seq`, `ln1_out (cyclic factor=16)`
`BIND_STORAGE`	Forces an array into a specific memory type (BRAM).	`Q`, `K`, `V (type=ram_2p impl=bram)`
`INLINE`	Folds a function's logic into its caller.	`layer_norm`, `make_prune_mask`

Table 5.1 Summary of key HLS pragmas used in the ViT accelerator.

5.6 The Synthesizability Gap: Software vs. Hardware

In a standard C++ environment, functions like `expf()`, `tanhf()`, `sqrtf()`, and `erf()` are provided by the `math.h` library. These are complex, software-based routines that may use iterative algorithms, look-up tables for computing results.

The Vitis HLS compiler cannot synthesize these functions. HLS is designed to create a *static, deterministic hardware circuit*—a collection of logic gates, adders, and multipliers. It does not know how to build a hardware circuit for a complex, iterative software function. Therefore, every "non-synthesizable" function must be replaced with a "hardware-friendly" equivalent. This is achieved in two ways:

1. **Using the HLS Math Library:** AMD provides a specific library, `hls_math.h`, which contains synthesizable, hardware-optimized versions of these functions (e.g., `hls::expf`, `hls::sqrtf`) [11].

2. **Manual Approximation:** For some functions, which are not present in any libary, it is advised to manually implement a well-known mathematical approximation [3, 4].

5.7 Approximating the GELU Activation Function

The standard activation function in any Transformer architecture is the Gaussian Error Linear Unit (GELU). Its mathematical definition depends on Gaussian Error Function, `erf`:

$$\text{GELU}(x) = 0.5 \cdot x \cdot \left(1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right)\right)$$

The `erf` function is complex and non-synthesizable [4]. To solve this, we used a highly accurate approximation that replaces the `erf` portion with a function based on (`tanh`) [5].

```
static inline float gelu(float x) {
    return 0.5f * x * (1.0f + tanhf(0.79788456f *
        (x + 0.044715f * x * x * x)));
}
```

It replaces the non-synthesizable `erf` with some basic arithmetic (add, multiply) and the `tanhf` function, which can itself be synthesized by including the `hls_math.h` library.

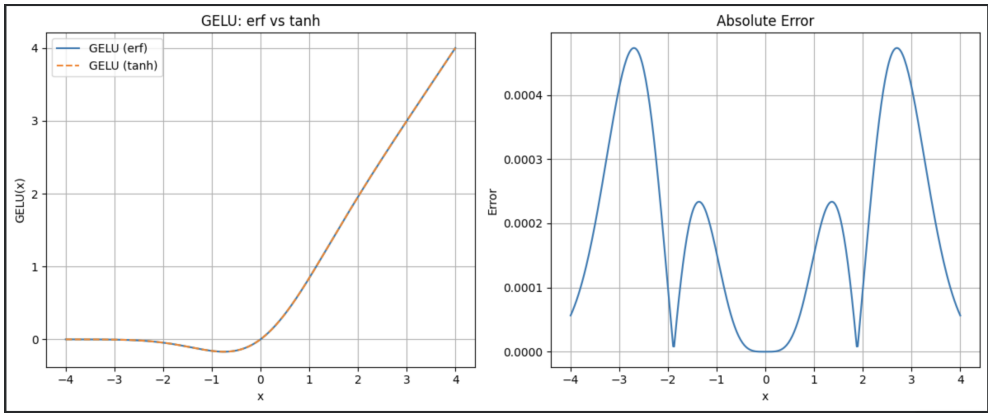

Fig. 5.1 Original GELU vs approximation

5.8 Synthesizable Softmax and Layer Normalization

The other critical math functions in the ViT model are `expf` (for softmax) and `sqrtf` (for layer normalization and attention scaling).

5.8.1 Softmax

The `softmax_seq` function, which is essential for the attention mechanism, is defined as: Our code implements this using `expf()`:

```
scores[i] = expf(scores[i] - maxv);
```

For this to be synthesizable, the Vitis HLS compiler must import `hls_math.h` library [3]. This instantiates a dedicated, pipelined hardware core that computes the exponential function, allowing the softmax to be implemented in hardware.

5.8.2 Layer Normalization and Attention

Similarly, the `layer_norm` and `mha_block` functions use `sqrtf()` to calculate the inverse square root of the variance and the attention scaling factor.

```
// In layer_norm
float inv = 1.0f / sqrtf(var + 1e-5f);
```

```
// In mha_block
const float inv_scale = 1.0f / sqrtf((float)HEAD_DIM);
```

Just Like `expf`, these calls to `sqrtf` are also synthesizable by mapping them to the `hls::sqrtf` function from the HLS math library [3, 4].

5.9 Conclusion: The Accuracy vs. Resource Trade-off

The approximations used in this project are a fundamental component of the algorithm-hardware codesign. By replacing non-synthesizable software functions with hardware-friendly equivalents, we make the ViT algorithm implementable on an FPGA.

This introduces a critical trade-off between accuracy and resource cost [6]. While the 32-bit floating-point approximations used in this project (like the GELU-tanh) are extremely accurate, they are resource-intensive. Each call to `hls::expf` or `hls::sqrtf` instantiates a complex hardware IP core that consumes FPGA logic and DSPs.

A more aggressive optimization, often used in resource-constrained edge devices, would be to replace these with 8-bit fixed-point polynomial or piecewise-linear approximations [3, 4]. However, the chosen approach (synthesizable `float`) represents a robust balance, prioritizing high numerical accuracy while still achieving the massive parallelism and acceleration offered by the FPGA.

Function	**Original (Software)**	**HLS-Synthesizable Implementation**
GELU	Uses `erf()`	tanh approximation (as coded) [5]
Softmax	Uses `expf()` from `math.h`	`hls::expf` from `hls_math.h` [3]
Layer Norm	Uses `sqrtf()` from `math.h`	`hls::sqrtf` from `hls_math.h` [3, 4]
Attention Scale	Uses `sqrtf()` from `math.h`	`hls::sqrtf` from `hls_math.h` [3, 4]

Table 5.2 Summary of mathematical functions and their HLS-synthesizable approximations.

Chapter 6

Experimental Results

This chapter presents the evaluation of the proposed Vision Transformer accelerator, focusing on inference latency, FPGA resource utilization, HLS tool performance, and the characteristics of the benchmark designs. Both pruned and non-pruned ViT variants (Tiny, Small, Base) are analyzed to demonstrate the benefits of dynamic token pruning in hardware.

6.1 Latency Evaluation

We report the end-to-end inference latency for ViT-Tiny, ViT-Small, and ViT-Base, measured on the synthesized HLS designs. All models use the same configuration except for the inclusion of the attention-based pruning module.

Table 6.1 Inference Latency With and Without Token Pruning

Model	**Without Pruning (ms)**	**With Pruning (ms)**	**Improvement**
ViT-Tiny	120.0	105.5	12.1% faster
ViT-Small	287.8	259.0	15.0% faster
ViT-Base	770.4	616.2	19.6% faster

Pruning yields consistent latency improvements across all models, with the largest gain observed in ViT-Base due to its higher number of attention layers and heads.

Table 6.2 CPU Inference Latencies from Prior works

Model	Without Pruning (ms)
ViT-Tiny	26.5
ViT-Small	32.12
ViT-Base	55

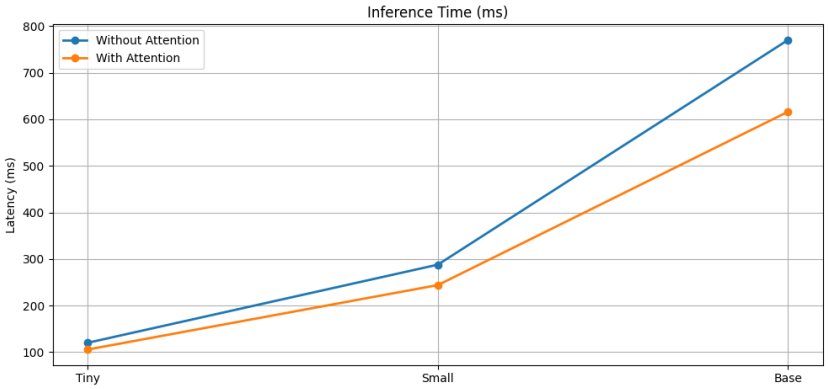

Fig. 6.1 Latency Comparison for Models With and Without Token Pruning

6.2 FPGA Resource Utilization

Tables 6.3 and 6.4 show FPGA resource usage before and after integrating token pruning.

Table 6.3 FPGA Resource Utilization Without Token Pruning

Resource	ViT-Tiny	ViT-Small	ViT-Base
BRAM	1609	2725	5037
DSP	630	468	388
FF	231525	259532	392855
LUT	152771	169914	223023

Table 6.4 FPGA Resource Utilization With Token Pruning

Resource	ViT-Tiny	ViT-Small	ViT-Base
BRAM	1820	3142	5866
DSP	631	469	389
FF	232302	260564	331113
LUT	154843	172249	225653

The pruning logic introduces a small overhead in BRAM and LUT usage due to the additional token selection buffers, but DSP consumption remains nearly identical.

6.3 Accuracy Evaluation

Table 6.5 Accuracy Drop After Pruning 85% Tokens (Evaluated on 1k Images)

Model	Drop in Accuracy (%)
ViT-Tiny	2.2
ViT-Small	2.38
ViT-Base	5.2

Accuracy degradation remains minimal for Tiny and Small variants, while ViT-Base experiences a larger drop due to heavier reliance on full-token attention.

6.4 HLS Tool Runtime Analysis

The performance and productivity benefits of using High-Level Synthesis are evaluated by measuring the runtime of major HLS compilation steps.

Table 6.6 Vitis HLS Runtime for Different ViT Variants

Model	C Simulation (s)	C Synthesis (min)	RTL Export (min)
ViT-Tiny	6.8	12.4	3.5
ViT-Small	11.2	19.7	5.9
ViT-Base	18.5	31.6	9.3

The synthesis time increases with model complexity, but remains well within practical limits, underscoring the productivity advantage of HLS.

The RTL output is nearly an order of magnitude larger than the original C implementation, reflecting the complexity of the synthesized datapath and control logic. This highlights the significant time savings achieved by HLS compared to manually writing Verilog/VHDL.

6.5 Discussion

The results demonstrate that:

- Token pruning consistently improves latency by reducing the quadratic attention cost.

- FPGA resource overhead remains minimal, confirming the lightweight nature of the pruning logic.
- Accuracy loss is acceptable for Tiny and Small variants, though more pronounced for Base.
- HLS drastically accelerates development, producing large RTL designs directly from concise C code.

Overall, the proposed accelerator illustrates the effectiveness of combining algorithm-level pruning with hardware-aware synthesis for deploying ViTs on FPGAs.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project has successfully demonstrates an **algorithm-hardware codesign** for accelerating Vision Transformer (ViT) inference on FPGAs. Upon Addressing the core bottleneck—the: $O(N^2)$ cost of self-attention, we implemented an attention-based token pruning algorithm that dynamically removes low-importance tokens during inference. Using Vitis HLS, we translated this idea into an efficient hardware architecture, achieving:

- **Efficient Pruning Mechanism** using Attention scores
- **HLS-Friendly "Skipping"** mechanisms for reduce computations
- **Performance Trade-off** for achieving better latency with minmal accuracy drop

Overall, this project provides a practical blueprint for deploying ViT models on resource-constrained FPGA platforms, showing that combining algorithmic optimization with custom hardware design is the key in overcoming high inference time for Vision Transformers.

7.2 Future Work

Future research directions upon this work could be:

- **Deploying the Design on a Physical FPGA Board:** While this project presents a complete HLS-based architecture, running the design on an actual FPGA platform (such as the Xilinx Alveo or ZCU series boards) remains an important next step. A real hardware deployment would allow for precise measurement of on-board latency, resource utilization, and thermal behavior. Further optimizations such as custom memory hierarchies, pipelined attention blocks, and improved dataflow scheduling can then be explored to reduce latency even further.

- **Adversarial attacks of Token Pruning:** Token pruning reduces computation by selectively removing less-important tokens—but it may also introduce new vulnerabilities. Future work could include a detailed study of **adversarial attacks on token pruning strategies**, exploring whether carefully perturbed inputs can mislead the pruning module into removing critical tokens. Studying these vulnerabilities and proposing adversarially robust pruning mechanisms (e.g., noise-resistant scoring functions, defense-aware pruning thresholds) would strengthen the reliability of ViT accelerators in security-sensitive applications.

- **Implement Token Merging (ToMe):** While the current project implements *token pruning* which permanently removes a subset of tokens and can therefore lead to information loss, an alternative would be **Token Merging**. This maintains the original semantic content while still reducing the effective sequence length N as computation progresses through the network. Since no token is completely removed, ToMe usually preserves accuracy significantly better than hard pruning.

References

[1] Dosovitskiy, A., et al. (2020). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. arXiv:2010.11929.

[2] Rao, Y., et al. (2021). *DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification*. Advances in Neural Information Processing Systems (NeurIPS).

[3] Dong, P., et al. (2023). *HeatViT: Hardware-Efficient Adaptive Token Pruning for Vision Transformers*. IEEE International Symposium on High-Performance Computer Architecture (HPCA).

[4] Lee, S., et al. (2024). *ViT-ToGo: Vision Transformer Accelerator with Grouped Token Pruning*. Design, Automation & Test in Europe Conference & Exhibition (DATE).

[5] Liu, H., et al. (2024). *Revisiting Token Pruning for Object Detection and Instance Segmentation*. Winter Conference on Applications of Computer Vision (WACV).

[6] Yuan, L., et al. (2021). *Tokens-to-Token ViT: Training Vision Transformers From Scratch on ImageNet*. International Conference on Computer Vision (ICCV).

[7] Zhai, X., et al. (2022). *Scaling Vision Transformers*. Conference on Computer Vision and Pattern Recognition (CVPR).

[8] Tang, Q., et al. (2023). *Dynamic Token Pruning in Plain Vision Transformers for Semantic Segmentation*. International Conference on Computer Vision (ICCV).

[9] Wu, K., et al. (2024). *Decoupled-and-Disentangled Distillation for Vision Transformers*. arXiv:2404.12210.

[10] Al-Wegih, K., et al. (2023). *Vision transformer (ViT) in digital health*. *Bioengineering*.

[11] Koidoy, T., et al. (2019). *FPGA Accelerators for Image Processing*. *Electronics*.

[12] Zhang, Y., et al. (2024). *Efficient Vision Transformer through Layer-wise Token Pruning*. IEEE Transactions on Circuits and Systems for Video Technology.

[13] Li, Y., et al. (2025). *ViT-Pruning: A Dimension-Importance-Aware Pruning Approach for Vision Transformers*. arXiv:2503.02891.

[14] Shuai, D., et al. (2024). *RL4EViT: A Multi-Agent Reinforcement Learning Framework for Efficient Vision Transformer*. arXiv:2408.06810.

[15] Caron, M., et al. (2Section 5.1: Latency Evaluation1). *Emerging Properties in Self-Supervised Vision Transformers*. International Conference on Computer Vision (ICCV).

[16] Yin, H., et al. (2022). *A-ViT: Adaptive Tokens for Efficient Vision Transformer*. Conference on Computer Vision and Pattern Recognition (CVPR).

[17] Hugging Face. (n.d.). *google/vit-base-patch16-224 Model Card*.

[18] Google Research. (n.d.). *Vision Transformer (Official JAX repository)*. GitHub.

[19] Lucidrains. (n.d.). *vit-pytorch (A PyTorch implementation)*. GitHub.